Project 3

12411723 杨嘉乐

2026 年 5 月 5 日

目录

1 引言与背景 **4**
1.1 性能优化方法论 4
1.2 报告结构概述 4

2 实验环境与基准配置 **5**
2.1 本地测试环境配置 5
2.2 云端服务器环境配置 5
2.3 基准库机制与配置准则 6
2.3.1 线程管理机制对比 6
2.3.2 编译期链接策略评估 7
2.4 性能基准与测试数据汇总 8
2.4.1 本地 6 核矩阵乘法执行时间与 GFLOPS(openblas-openmp) 8
2.4.2 本地 16 核矩阵乘法执行时间与 GFLOPS (openblas-openmp) . . . 8
2.4.3 云服务器 56 核矩阵乘法执行时间与 GFLOPS(openblas-openmp) . 9

3 矩阵乘法核心实现与版本演进 **9**
3.1 内存布局与数据结构定义 9
3.2 基础并行化实现 10
3.2.1 V1: 朴素三重循环实现 10
3.2.2 V2: 线程级并行与访存重排 11
3.2.3 V3: 基础向量化运算引入 12
3.3 缓存感知与微内核优化 13
3.3.1 V4: 分块机制与指令级并行展开 13
3.3.2 分块尺寸参数调优实验 14
3.3.3 V5: 寄存器阻塞 16
3.3.4 微内核尺寸性能对比 17

3.3.5 V6: 二级缓存对齐与矩阵面板打包 18
3.4 架构级深度调优 20
3.4.1 V7-V10: 连续性访存、预取与 SIMD 指令替换 21
3.4.2 `matmul_improved` 架构选择与参数决策 24

4 OPENBLAS 分析 **24**
4.1 运行时调用链 24
4.2 数据流模型与多级缓存映射机制 27
4.3 本阶段实现与标杆库的工程差异对比 28

5 JIT 版本 **28**
5.1 操作系统与物理内存调优 28
5.2 多核并发调度与同步消除 29

6 最终性能测试与对标验证 **29**
6.1 全规模矩阵执行耗时与浮点运算性能表现 29
6.2 对标基准库加速比分析 29

7 低时间复杂度算法 **30**
7.1 Strassen 算法 ($O(N^{2.807})$) 30
7.1.1 分治与子块定义 30
7.1.2 构造 7 个中间乘积项 ($P_1 \sim P_7$) 30
7.1.3 结果子块的组合还原 31
7.1.4 复杂度与工程权衡 31
7.2 Winograd 变体 31
7.3 理论极限：Laser Method 与 Coppersmith-Winograd 32
7.4 不同算法性能实测与复杂度验证 32
7.4.1 编译环境说明 32
7.4.2 实验数据汇总 32

8 GPU 加速优化与性能分析 **32**
8.1 实验环境说明 32
8.2 CUDA 核心优化实现 33
8.2.1 共享内存分块 (Shared Memory Tiling) 33
8.2.2 寄存器分块 (Register Tiling) 33
8.3 编译与运行指令 34
8.4 测试结果与分析 34

9 ARM 架构下的高性能 GEMM 移植与极限优化 **35**
9.1 实验环境与硬件规格 35
9.2 从 x86 AVX2 到 ARM NEON 的 SIMD 矢量化重构 36
9.2.1 6×16 内核的分配 36
9.3 实验结果与性能评估 37

10 未来工作与不足 **37**

11 总结与心得 **38**

1 引言与背景

1.1 性能优化方法论

在开始我的 project 前,我首先想引用 Algorithmica 网站上的 Algorithms for Modern Hardware 这本书里的这样几句话:

When I began learning how to optimize programs myself, one big mistake I made was to rely primarily on the empirical approach. Not understanding how computers really worked, I would semi-randomly swap nested loops, rearrange arithmetic, combine branch conditions, inline functions by hand, and follow all sorts of other performance tips I' ve heard from other people, blindly hoping for improvement.

Unfortunately, this is how most programmers approach optimization. Most texts about performance do not teach you to reason about software performance qualitatively. Instead they give you general advice about certain implementation approaches —and general performance intuition is clearly not enough.

It would have probably saved me dozens, if not hundreds of hours if I learned computer architecture *before* doing algorithmic programming. So, even if most people aren' t *excited* about it, we are going to spend the first few chapters studying how CPUs work and start with learning assembly.

计算机体系结构 - Algorithmica — Computer Architecture - Algorithmica

罗马不是一日建成的，优化不是一步到位的。本次 project，我在通过 v1 到 v10，运用 OpenMP，SIMD，分块，寄存器，NUMA，针对本机的 Intel 优化，在每一次优化中学习原理。由于我在做 project 之前没有相关的知识储备，于是在开始前我先阅读了上面的这本书，第九章RAM & CPU Caches和第十章SIMD Parallelism对本次 project 帮助很大。

每做一个优化，都需要明白原理，这样优化解决了什么问题？和之前的版本有什么区别？现在的优化带来了什么新的问题？可以怎么通过新的版本解决？

1.2 报告结构概述

由于本机内存有限,我的 project 分为两部分,第一部分是在本地的 Intel(R) Core(TM) Ultra 7 255H 去做 16384 * 16384 及以下的矩阵计算。第二部分在 CS308 计算机视觉课给的云服务器 (Intel(R) Xeon(R) CPU E5-2680 v4 @ 2.40GHz) 中去进行 64k * 64k 的矩阵计算。第三部分是 GPU 的计算 (NVIDIA TITAN V) 和 cuBLAS 的基准。第四部分是阿里云服务器的 arm 架构 (virt-rhel7.6.0 CPU @ 3.0GHz) 的实验。

对于 OpenBLAS，这是一个庞大的库。他很完备，不过我们的优势在于我们充分了解自己的硬件，以及我们可以对线程进行限制。

2 实验环境与基准配置

2.1 本地测试环境配置

硬件维度	具体参数
处理器型号	Intel(R) Core(TM) Ultra 7 255H
核心拓扑	16 Cores, 1 Thread / Core
L1 数据缓存	48 KiB / Core
L2 缓存	3 MiB / Core
L3 缓存	24 MiB
关键指令集	AVX2, FMA

软件维度	具体配置
操作系统	Ubuntu 24.04.3 LTS (WSL2)
资源限制	16 Cores, 16GB RAM
编译器	GCC 13.3.0
基准对标库	OpenBLAS 0.3.26

有 6 个 P 核，10 个 E 核

2.2 云端服务器环境配置

参数名称	详细规格
处理器型号	Intel(R) Xeon(R) CPU E5-2680 v4 (Broadwell 架构)
主频 (Base/Max)	2.40 GHz / 3.30 GHz
架构	x86_64 (64-bit)
物理插槽 (Sockets)	2
物理核心数 (Cores)	28 (每个插槽 14 核)
逻辑线程数 (CPUs)	56 (支持超线程 HT)
超线程状态	已开启 (2 threads per core)

缓存级别	总容量 (所有核心总和)	每个核心分配
L1d (数据)	896 KiB (28 实例)	32 KiB
L1i (指令)	896 KiB (28 实例)	32 KiB
L2 缓存	7 MiB (28 实例)	256 KiB
L3 缓存	70 MiB (2 实例)	35 MiB

参数名称	详细规格
NUMA 节点数	2 (双路系统)
节点 0 CPU 列表	0-13, 28-41 (共 28 逻辑核)
节点 1 CPU 列表	14-27, 42-55 (共 28 逻辑核)
内存访问模式	跨节点访问延迟高于本地访问 (需使用 numactl –interleave)

指令集	作用说明
AVX / AVX2	支持 256 位向量运算，单指令处理 8 个单精度浮点数
FMA	核心算力来源。支持乘加同步完成 ($a \times b + c$)，算力翻倍
VT-x	硬件辅助虚拟化支持
CAT_L3	缓存分配技术 (Cache Allocation Technology)，支持 L3 隔离

2.3 基准库机制与配置准则

在 Ubuntu 系统中，通过 update-alternatives 机制，我们可以选择不同的 OpenBLAS 实现方案。关于 openblas-pthread 与 openblas-openmp 两种版本的库，效果不一样，本机实测 openmp 会快一些。

2.3.1 线程管理机制对比

openblas-pthread：- 实现机制：该版本使用原生 POSIX 线程（Pthreads）管理并发。OpenBLAS 内部维护着一套私有的、独立于编译器的线程池。- 控制变量：主要受 OPENBLAS_NUM_THREADS 环境变量控制。- 行为特征：由于它不感知外层的并行环境，当在用户自定义的 OpenMP 并行块中调用该库时，会产生“嵌套线程”现象。

openblas-openmp：- 实现机制：该版本底层直接链接至 OpenMP 运行时库（如 libgomp）。它将线程管理任务委托给系统的 OpenMP 调度器。- 控制变量：完全服从标准 OMP_NUM_THREADS 环境变量。- 行为特征：它具备“线程感应”能力。当检测到外层已经存在 OpenMP 环境时，它会倾向于复用当前线程或遵循嵌套并行规则，从而实现更和谐的资源分配。

维度	openblas-pthread	openblas-openmp
线程池管理	库函数自建线程池，不与调用者通信	与调用者（GCC OpenMP）共享运行时
过度订阅风险	极高。$N_{OMP} \times M_{Pthread}$ 线程爆炸	低。线程总数受 OpenMP 统一管控
上下文切换开销	高。大量线程抢占导致 CPU 缓存失效	低。线程数与物理核心数易于对齐
性能稳定性	在单线程调用时极稳；多层嵌套时崩盘	在复杂并行逻辑中表现更稳健、高效
本实验适用性	需额外小心 OPENBLAS_NUM_THREADS 设置	首选方案，确保基准测试纯净度

2.3.2 编译期链接策略评估

使用 openblas-openmp 作为标准

- 编译指令：

```
gcc -Ofast -march=native -mfma -mavx2 -fopenmp main.c matmul.c -
    o gemm_bench -lblas
```

- 使用 -lblas 链接经由 alternatives 路由至 openblas-openmp 的动态库。（因为我们在 update-alternatives 这里选择了 openmp，所以不需要 -lopenblas 显式调用）

可以选择将 6 个计算任务强行绑定在 P 核上的一些约束：

- export OMP_NUM_THREADS=6
- OMP_PROC_BIND=close 与 OMP_PLACES="{0}:6:1"
- 使用 taskset -c 0-5 启动

如果使用满核（16 个）

- export OMP_NUM_THREADS=16
- OPENBLAS_NUM_THREADS=16
- OMP_PROC_BIND=close
- OMP_PLACES=cores
- 使用 taskset -c 0-15 启动

2.4 性能基准与测试数据汇总

2.4.1 本地 6 核矩阵乘法执行时间与 GFLOPS(openblas-openmp)

优化版本	16 x 16		128 x 128		1024 x 1024		8192 x 8192	
	Time (s)	GFLOPS	Time (s)	GFLOPS	Time (s)	GFLOPS	Time (s)	GFLOPS
V1 (Plain)	**0.000002**	**4.10**	**0.000735**	**5.71**	2.175703	0.99	-	-
V2 (OMP+i-k-j)	0.000655	0.01	**0.000242**	**17.33**	0.018236	117.76	-	-
V3 (SIMD+OMP)	**0.000002**	**4.10**	**0.000084**	**49.93**	0.031926	67.26	-	-
V4 (SIMD+OMP+Tiling)	**0.000003**	**2.73**	**0.000144**	**29.13**	0.020281	105.89	-	-
V5 (Micro-kernel)	**0.000002**	**4.10**	**0.000032**	**131.07**	**0.005126**	**418.94**	-	-
V6 (Micro-kernel+packingB)	0.000643	0.01	**0.000056**	**74.90**	**0.005047**	**425.50**	2.698930	407.39
V7 (Micro-kernel+packingAB)	0.000115	0.07	**0.000062**	**67.65**	0.010223	210.06	2.458593	447.21
V8 (Lunar-Optimized+K-Unroll+Prefetch)	**0.000004**	**2.05**	**0.000250**	**16.78**	0.009228	232.71	**2.308162**	**476.36**
V9 (Lunar-Max-L2+Large-Tiling)	0.001290	0.01	**0.000036**	**116.51**	0.019243	111.60	2.405235	457.13
V10 (Symmetric-Packing+Broadcast)	**0.000007**	**1.17**	**0.000034**	**123.36**	**0.008729**	**246.02**	**2.309331**	**476.12**
OpenBLAS (Reference)	0.000053	0.15	0.003317	1.26	0.008963	239.60	2.334064	471.07

2.4.2 本地 16 核矩阵乘法执行时间与 GFLOPS (openblas-openmp)

下表展示了在本地 16 线程满载环境下，各优化版本的执行时间（秒）与对应的浮点运算吞吐量（GFLOPS）。加粗的数据表示在该矩阵规模下，该版本的表现成功超越了 OpenBLAS 基准。

优化版本	16 x 16		128 x 128		1024 x 1024		8192 x 8192		16384 x 16384	
	Time (s)	GFLOPS	Time (s)	GFLOPS	Time (s)	GFLOPS	Time (s)	GFLOPS	Time (s)	GFLOPS
V1 (Plain)	**0.000001**	**8.19**	**0.000846**	**4.96**	1.954681	1.10	-	-	-	-
V2 (OMP+i-k-j)	0.015489	0.00	**0.001906**	**2.20**	**0.011636**	**184.55**	17.521575	62.75	166.267461	52.90
V3 (SIMD+OMP)	0.021487	0.00	**0.005141**	**0.82**	**0.017728**	**121.13**	23.747123	46.30	211.654556	41.56
V4 (SIMD+OMP+Tiling)	0.013172	0.00	**0.002580**	**1.63**	**0.013214**	**162.52**	10.987217	100.07	144.490792	60.88
V5 (Micro-kernel)	0.012581	0.00	**0.004547**	**0.92**	**0.009626**	**223.09**	5.536946	198.58	66.129406	133.01
V6 (Micro-kernel+packingB)	0.006600	0.00	**0.006902**	**0.61**	**0.012141**	**176.88**	**1.391692**	**790.05**	**10.707752**	**821.47**
V7 (Micro-kernel+packingAB)	0.005088	0.00	**0.002991**	**1.40**	0.031152	68.94	1.560106	704.77	**11.587811**	**759.08**
V8 (Lunar-Optimized+K-Unroll+Prefetch)	0.005259	0.00	**0.006683**	**0.63**	0.026733	80.33	1.632152	673.66	**11.908975**	**738.61**
V9 (Lunar-Max-L2+Large-Tiling)	0.008670	0.00	**0.006880**	**0.61**	0.026255	81.79	1.836269	598.77	13.031667	674.98
V10 (Symmetric-Packing+Broadcast)	0.003762	0.00	**0.002315**	**1.81**	**0.012650**	**169.76**	**1.319376**	**833.36**	**9.860266**	**892.07**
OpenBLAS (Reference)	0.000139	0.06	0.008709	0.48	0.024987	85.94	1.406089	781.96	11.995189	733.30

2.4.3 云服务器 56 核矩阵乘法执行时间与 GFLOPS(openblas-openmp)

矩阵规模 (N x N)	指标	Plain (V1)	Improved	JIT	OpenBLAS
16 x 16	Time (s)	**0.000011**	0.000055	0.000012	0.000079
	GFLOPS	**0.75**	0.15	0.69	0.10
128 x 128	Time (s)	0.002228	0.000284	0.000289	**0.000239**
	GFLOPS	1.88	14.77	14.50	**17.54**
1024 x 1024	Time (s)	2.293088	0.004810	**0.004444**	0.004692
	GFLOPS	0.94	446.45	**483.22**	457.68
8192 x 8192	Time (s)	Skipped	0.886784	**0.810066**	0.908520
	GFLOPS	-	1239.89	**1357.31**	1210.22
16384 x 16384	Time (s)	Skipped	6.964683	6.833305	**6.681441**
	GFLOPS	-	1262.96	1287.24	**1316.50**
32768 x 32768	Time (s)	Skipped	52.412215	54.129841	**41.859317**
	GFLOPS	-	1342.60	1300.00	**1681.08**
65536 x 65536	Time (s)	Skipped	408.587450	424.037382	**339.170381**
	GFLOPS	-	1377.80	1327.60	**1659.79**

3 矩阵乘法核心实现与版本演进

3.1 内存布局与数据结构定义

我们首先定义结构体：

```
typedef struct {
    size_t rows;
    size_t cols;
    float *data;
} Matrix;
```

按照 project3，我们定义的 Matrix 结构体采用了行主序的存储方式。在 C 语言中，虽然我们在逻辑上操作的是 $M \times K$ 和 $K \times N$ 的二维矩阵，但为了保证内存分配的连续性，矩阵在物理内存中是被展平为一个一维数组的。

其寻址公式为：

$$Address = Base + (RowIndex \times Columns + ColIndex) \times sizeof(float)$$

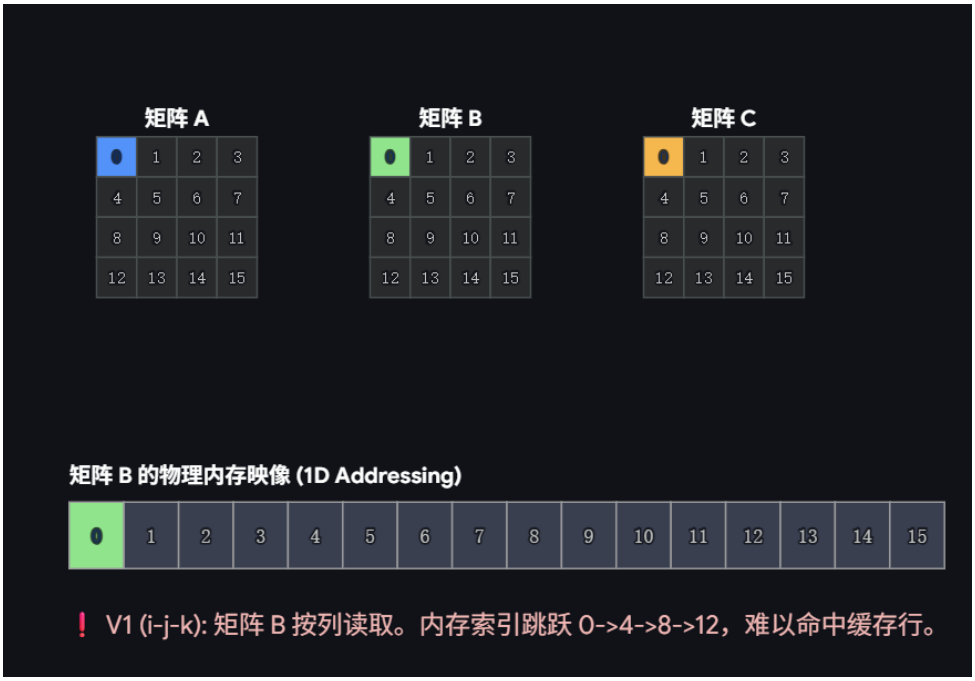
3.2 基础并行化实现

3.2.1 V1: 朴素三重循环实现

v1 就是朴素的 i-j-k 实现:

```
for (size_t i = 0; i < A->rows; i++)
    {
        for (size_t j = 0; j < B->cols; j++)
        {
            float sum = 0.0f;
            for (size_t k = 0; k < A->cols; k++)
            {
                sum += A->data[i * A->cols + k] * B->data[k * B->cols
                    + j];
            }
            C->data[i * C->cols + j] = sum;
        }
    }
```

- 对于 A 矩阵 (i * A->cols + k): 随着 k 以步长为 1 递增，程序在物理内存中是连续、顺序读取 A 矩阵元素的。- 对于 B 矩阵 (k * B->cols + j): 随着 k 递增，k 每次都要乘以 B->cols。这意味着在物理内存中，程序在进行大跨度的跳跃访问。



在 V1 的最内层循环 k 中，计算 $C[i, j]$ 时，程序需要沿着矩阵 B 的列方向向下遍历。

1. 跨步惩罚 (Stride Penalty)：每次 k 增加 1，内存地址就会跳跃一整行（1024 个 float，即 4 KiB 的物理距离）。

2. 缓存行的浪费：现代 CPU 读取内存的最小单位是缓存行（Cache Line），通常是 64 字节（16 个 float）。当你读取 B[0,0] 时，CPU 会把同一行的 B[0,1] 到 B[0,15] 全加载进来。但在 V1 中，你的下一步是去读 B[1,0]，刚才加载的 15 个数全被浪费了。

3. L1 容量的致命越界：当 n>1024 时，为了读完矩阵 B 的一整列（1024 个元素），CPU 被迫加载了 1024 个毫无关联的缓存行。这需要占用的缓存空间是：1024 × 64 Bytes = 64 KiB。

通过查询，L1d 只有 48 KiB。这意味着，当循环还没有走到矩阵底部时，L1 缓存就已经被撑爆了。最先加载的列顶部数据会被迫踢出 L1。等到你计算下一个结果 $C[i, j+1]$ 再次需要读取这些数据时，它们早已不在 L1 中了。所以这能解释，为什么 n>1024 时，v1 的速度显著变慢。

3.2.2 V2: 线程级并行与访存重排

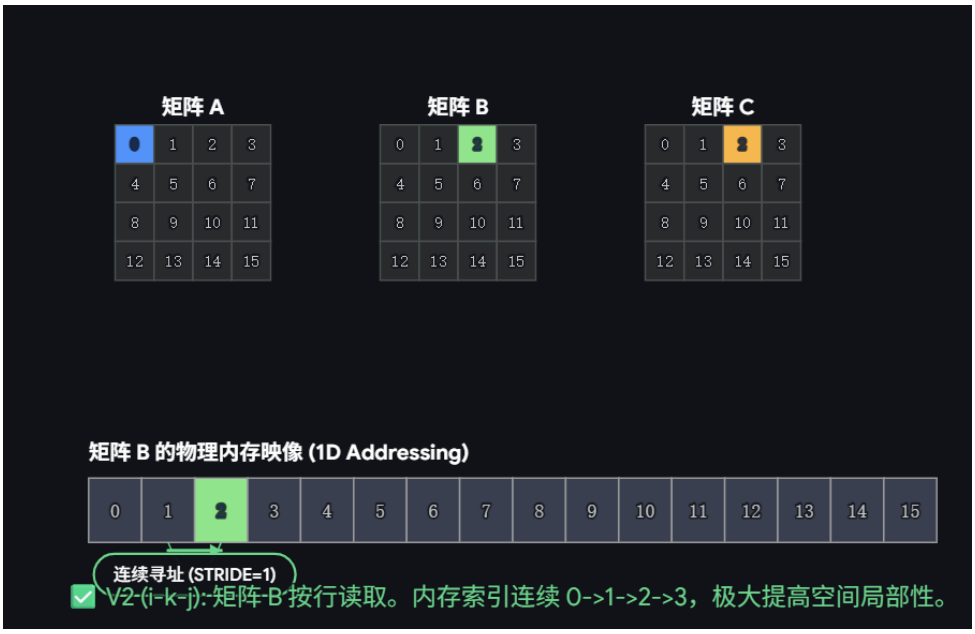
v2 就是引入了 OpenMP 并且重排循环为 i-k-j 的实现：

```
#pragma omp parallel for
for (size_t i = 0; i < A->rows; ++i)
{
    for (size_t k = 0; k < A->cols; ++k)
    {
        float temp = A->data[i * A->cols + k];
        for (size_t j = 0; j < B->cols; ++j)
        {
            C->data[i * C->cols + j] += temp * B->data[k * B->cols +
                j];
        }
    }
}
```

- 对于 B 矩阵 (k * B->cols + j)：随着 j 以步长为 1 递增，程序在物理内存中是连续、顺序读取 B 矩阵元素的。- 对于 C 矩阵 (i * C->cols + j)：同理，随着 j 递增，程序在物理内存中也是连续、顺序写入 C 矩阵元素的。- 对于 A 矩阵 (i * A->cols + k)：由于在最内层 j 循环中 i 和 k 都是不变的，它被提取成了局部变量 temp。这意味着它直接被放在了光速的 CPU 寄存器里，彻底消灭了内层循环的重复读取。

在 V2 的最内层循环 j 中，更新 C[i, j] 时，程序改为沿着矩阵 B 和 C 的行方向向右遍历。每次 j 增加 1，内存地址仅仅前进 1 个 float 的 4 个字节，不再有整行的跳跃。

引入 openmp：最外层加了 #pragma omp parallel for，把矩阵的不同行完美平分给了 16 个纯物理核同时跑，且互相写入互不干涉。



3.2.3 V3: 基础向量化运算引入

SIMD 和 OpenMP 都是并行，但并不相同：

- SIMD (AVX/FMA) 解决的是数据级并行 (Data-Level Parallelism)。它负责把单颗核心的算力压榨到极限 (1 核当 8 核用)。- OpenMP 解决的是线程级并行 (Thread-Level Parallelism)。它负责把任务分发给多颗物理核心（16 核一起上）。

在 lscpu 中可以查到 Flags 的信息，其中包括 avx fma, 编译时加上-mavx2 -mfma 就可开启。

学写 SIMD：C 语言的内置函数（Intrinsics）有着严谨的命名公式，这样一下就能看懂：_mm< 位宽 >_< 操作名 >_< 数据类型 >

v3 代码：

```
#pragma omp parallel for
for (size_t i = 0; i < A->rows; ++i)
{
    for (size_t k = 0; k < A->cols; ++k)
    {
        float a_val = A->data[i * A->cols + k];
        __m256 va = _mm256_set1_ps(a_val);
        size_t j_limit = (B->cols / 8) * 8;
        size_t j = 0;
        for (; j < j_limit; j += 8)
        {
            __m256 vb = _mm256_loadu_ps(&B->data[k * B->cols + j]);
            __m256 vc = _mm256_loadu_ps(&C->data[i * C->cols + j]);
            vc = _mm256_fmadd_ps(va, vb, vc); // FMA 指令加速
```

```
            _mm256_storeu_ps(&C->data[i * C->cols + j], vc);
        }
        // 处理尾部不满足 8 个的部分 (j 从 j_limit 继续往后走)
        for (; j < B->cols; ++j)
        {
            C->data[i * C->cols + j] += a_val * B->data[k * B->cols +
                j];
        }
    }
}
```

- 1 AVX 向量化运算 (SIMD)：引入 256 位宽的 YMM 寄存器（代码中的 ___m256）。单条指令可同时吞吐 8 个单精度浮点数（float），使得内层循环步长直接跃升为 j += 8。- 2 硬件级数据广播 (Broadcasting)：通过 _mm256_set1_ps 指令，将矩阵 A 的标量值在硬件层面瞬间复制 8 份并填满向量寄存器，为后续的并行乘法做准备。- 3 非对齐安全访存 (Unaligned Memory Access)：采用 _mm256_loadu_ps 和 _mm256_storeu_ps 一次性读写 8 个浮点数。后缀 u (unaligned) 完美规避了矩阵列数非 8 整数倍时跨块读取可能引发的内存段错误。- 4 FMA 融合乘加 (Fused Multiply-Add)：_mm256_fmadd_ps 指令调用 CPU 专用计算单元，在一个时钟周期内将乘法和加法合并执行。

但是，当-Ofast 时，v3 是没有 v2 快的，用-fopt-info-vec-optimized 编译选项进行底层探针分析，发现 GCC 在 -Ofast 和 -march=native 优化级别下，已经完美地将 V2 版本的 i-k-j 内层循环自动向量化为 32-byte (256-bit) 的指令。V3 强行插入的 _mm256_loadu_ps 内置函数限制了编译器的发挥空间。

那么 v3 的瓶颈在哪里？当矩阵过大时，数据根本塞不进缓存，CPU 算得再快，也必须等主存把下一批数据送过来。在内存带宽成为瓶颈时，盲目提升计算吞吐量毫无意义。(在 project2 中我们能够发现，制约性能的关键不一定在于 cpu 后端的计算，前端去读取写入数据也是关键的性能点)

3.3 缓存感知与微内核优化

3.3.1 V4: 分块机制与指令级并行展开

为什么要分块？不分块，主要的数据来源是主存 RAM，每个矩阵元素从主存加载到 CPU 后通常只参与极少数次运算。而分块，主要的数据来源就是 L1/L2 缓存，一个加载到缓存的 B 块可以与 A 块中的多行进行连续运算。理论上，分块将主存访存次数从 $O(N^3)$ 降低到了 $O(N^3/BLOCK_SIZE)$，极大程度地缓解了对内存带宽的依赖。

所以，分块的目的是将数据“锁”在高速缓存中，提高缓存命中率，提高数据复用性，以提高整个程序的性能。

V4 在 V3 的基础上引入了两个关键的工程优化：分块 (Tiling) 和手动循环展开

(Manual Unrolling)。

- 分块循环控制 (bi, bk)：

```
#pragma omp parallel for collapse(2)
for (size_t bi = 0; bi < A->rows; bi += BLOCK_SIZE)
    for (size_t bk = 0; bk < A->cols; bk += BLOCK_SIZE)
```

通过引入 bi 和 bk 两层外循环，算法不再是一次性扫描整行数据，而是将任务切割成大小为 BLOCK_SIZE x BLOCK_SIZE 的局部任务。collapse(2) 指令将这两个维度合并为一个任务池，更多的任务总数使得 OpenMP 调度器能更灵活地分配负载。当个别物理核因系统干扰出现瞬时波动时，其他核心能通过处理更多小规模任务来弥补性能缺口，从而实现了更优的负载均衡 (Load Balancing)。

- 4 路向量化展开 (j += 32)：

```
for (; j + 31 < B->cols; j += 32) {
    __m256 vb0 = _mm256_loadu_ps(&B->data[k * B->cols + j]);
    // ... 加载 vb1, vb2, vb3
    // ... 执行 4 路 fmadd
}
```

现代 CPU 执行 vfmadd 指令存在固有的时钟周期延迟。V4 采用 4 路循环展开，同时操作 vc0 至 vc3 四组独立寄存器。由于这四组运算相互之间不存在数据依赖（Data Dependency），CPU 的乱序执行引擎可以实现指令级并行 (ILP)。通过在单次循环中同时发射多条乘加指令，CPU 成功利用后续指令的执行“掩盖”了前序指令的计算延迟，大幅提升了核心流水线的吞吐效率。

特性	V3 (SIMD + OpenMP)	V4 (Tiling + Unrolling)
访存模式	$i-k-j$ 顺序扫描，击穿缓存	分块扫描 (Blocked)，锁定在 L1/L2 缓存内
并行粒度	仅最外层行级并行 (Row-level)	块级并行 (Tile-level)，负载更均衡
指令密度	标量循环步长为 8，单路计算	4 路循环展开，步长为 32，指令密度更高
数据重用率	低（读一次算一次）	高（缓存内反复重用）
瓶颈点	内存带宽 (Memory Bound)	计算吞吐 (Compute Bound)

3.3.2 分块尺寸参数调优实验

分块分多大？为了寻找最佳的 BLOCK_SIZE (BS)，本实验在不同矩阵规模下，线程数 16，进行了多组横向对比测试。实测结果如下表所示：

不同分块大小在各规模下的性能表现 (单位: GFLOPS)

矩阵规模 (N)	BS=32	BS=64	BS=128	BS=256	BS=512	最佳分块
1024	155.25	130.23	172.45	142.72	81.59	BS = 128
2048	236.53	232.11	210.31	235.06	160.28	BS = 32 / 256
4096	252.47	209.76	186.17	150.98	81.70	BS = 32
8192	192.71	159.92	109.00	72.91	57.62	BS = 32

有几个因素：1. 硬件限制 2. 矩阵规模 3. 寄存器与指令集 (Registers & ISA) 4. 线程核数与分配 5.TLB (寻址转换) 与内存页

1. **硬件缓存**
   这是 8192 规模下性能急剧下滑的根本原因。(说明 v4 还是不够优秀，这会在后面的版本继续优化。)
   - L1d 适配策略：本机的私有 L1d 缓存为 48 KiB。
   - 当 $BS = 32$ 时：子块内存占用为 $32 \times 32 \times 4B = 4KiB$。该大小仅占 L1d 的 1/12，数据都能放在 L1 缓存中。
   - 当 $BS = 128$ 时：子块占用 64 KiB>L1d 的 48 KiB。在大规模矩阵（8192）下，我认为是因为数据在 L1 和 L2 之间搬运导致的性能下降。
2. **并行与负载均衡**
   数据解释了为什么在 1024 规模下，128 比 32 更好。
   - 任务调度开销：在 1024 规模下，若 $BS = 32$，会产生 $(1024/32)^2 = 1024$ 个小任务。虽然并行度高，但由于每个任务计算时间极短，OpenMP 管理线程、分配任务的软件开销占比过高。
   - $BS = 128$ 在 1024 规模下产生 64 个任务。这对于 16 个物理核心而言，既能保证每个核平均分到 4 个任务实现负载均衡，又通过减少任务总数降低了调度成本，因此跑出了 172 GFLOPS 的峰值。
3. **矩阵规模 (N) 与“内存墙”效应**
   - 中等规模 (2048)：由于总体数据量约为 48 MiB，仍可获得 24 MiB L3 缓存的带宽支撑，因此分块尺寸在 32、128、256 时性能表现相对均衡。
   - 大规模 (8192)：总体数据量达到 768 MiB，已大幅超出高速缓存的物理容量上限。此时分块尺寸成为控制访存带宽开销的关键手段。分块尺寸为 32 时，分块小，缓存命中率高。大分块就要去更多访问主存 (因为 L3 都放不下)。
4. **TLB 与内存页效率**
   大分块（如 512，占用 1 MiB 内存）在 8192 维度性能不佳。除了缓存击穿外，还有严重的 TLB (Translation Lookaside Buffer) Miss。（原理 project2 有过讲解）512 子块涉及的物理页表过多，导致 CPU 在地址转换上花费了大量周期，性能不佳。

所以 v4 最终选择了 BS=32。当然，这是根据我的设备特性的选择。上面的结论是在同样的实验条件，只改变了分块大小得出的。所以通过这个小实验可以发现不存在一个“放之四海而皆准”的固定分块大小。所以，当 AI 给你一个分块大小，这只是一个参考，AI 不能代替你跑实验，谨慎使用。

总结一下，对于计算量较小的任务，应选择稍大的分块（如 128），以权衡缓存复用与调度开销。对于计算量巨大的任务，必须回归“适配 L1d”原则（如 32），通过牺牲一部分任务调度的平滑度，换取最底层的访存带宽保护。

3.3.3 V5: 寄存器阻塞

V5 版本引入了工业级数学库（如 OpenBLAS, BLIS）的核心技术：微内核（Micro-kernel）。对于数据复用，从 v4 的缓存层面到了 v5 的寄存器层面。

1. 寄存器阻塞 (Register Blocking) V5 构建了 6×16 的内核。我们的 cpu 有 16 个 YMM 寄存器，我们这样分配：

- **结果累加器 (C)：** 占 12 个寄存器（6 行 $\times$ 2 向量）。
- **数据加载器 (B)：** 占 2 个寄存器（一次读取 16 列）。
- **标量广播器 (A)：** 占 1 个寄存器（用于存放 a_{ik}）。
- **预算总计：** 15 个寄存器。该设计在不触发寄存器溢出（Spilling）的前提下，将 B 矩阵的重用率提升了 6 倍。
- **作用：** 在 CPU 最内层的计算循环中，手动指定一组向量寄存器（如代码中的 `c0` 到 `cb`）来驻留 C 矩阵的中间结果，并让从 B 矩阵加载的数据在寄存器中被多次复用。

A. 申请寄存器

```
// 申请 12 个 YMM 寄存器，把 C 矩阵的一个 6x16 块死死锁在手里
__m256 c0 = _mm256_setzero_ps(), c1 = _mm256_setzero_ps(); // 第 0 行
__m256 c2 = _mm256_setzero_ps(), c3 = _mm256_setzero_ps(); // 第 1 行
// ... 连续定义到 c9, ca, cb（共 12 个）
```

- **实现逻辑：** 这 12 个变量在编译后会直接对应 CPU 内部的物理寄存器。在整个 k 循环结束前，它们绝不会回写内存。

B. 实现“1 换 6”的高倍复用 (The Core FMA)

```
for (size_t k = 0; k < K; ++k) {
    // 1. 从打包好的 B 中读取 16 个元素 (2 个向量)
    __m256 b0 = _mm256_loadu_ps(B + k * ldb);
    __m256 b1 = _mm256_loadu_ps(B + k * ldb + 8);

    // 2. 核心复用: 加载 A 的 1 个元素, 广播给刚才拿到的 16 个 B 元素
    __m256 va0 = _mm256_set1_ps(A[0 * lda + k]);
    c0 = _mm256_fmadd_ps(va0, b0, c0);
    c1 = _mm256_fmadd_ps(va0, b1, c1);

    // 3. 复用: 刚才拿到的 b0, b1 还没丢, 继续跟 A 的第 1 行算
    __m256 va1 = _mm256_set1_ps(A[1 * lda + k]);
    c2 = _mm256_fmadd_ps(va1, b0, c2);
    c3 = _mm256_fmadd_ps(va1, b1, c3);
    // ... 这一直持续到 va5 与 ca, cb 的运算
}
```

- **实现逻辑：**你会发现，在一次 k 循环中，b0 和 b1 只执行了 1 次 Load，却参与了 6 次 FMA 计算。这就是代码层面的 “减少访存” ——我们将访存频率压低到了计算频率的 1/6。

那这解决了 v4 的什么问题：L1 缓存里，CPU 每次执行 load 指令依然需要 4-5 个时钟周期。如果你的计算指令和 load 指令比例是 $1:1$，CPU 就会因为频繁去 L1 拿货而导致流水线停顿。通过 6×16 阻塞，你每从 B 拿一次数，可以连续做 6 次 FMA 计算。这让计算/访存比大幅提升，简单概括就是让 CPU 不用等。

3.3.4 微内核尺寸性能对比

2. 数据打包 (Data Packing) 针对大规模矩阵，V5 引入了 `pack_B_panel` 模块。其作用是将跨行、不连续的 B 矩阵子块重新拷贝到一片物理连续的缓冲区中。这消除了内存步长（Stride）带来的 TLB Miss。

A. 将 “跳跃” 转为 “连续” (The Packing Step)

```
static inline void pack_B_panel(size_t k_len, size_t n_len, const
   float *src, size_t ld_src, float *dest) {
    for (size_t k = 0; k < k_len; ++k) {
        memcpy(dest + k * n_len, src + k * ld_src, n_len * sizeof(
           float));
    }
}
```

- **实现逻辑：**原本 B 矩阵两行之间隔了 N 个元素（可能是 8192），现在被 memcpy 强行拉到了彼此相邻的位置。

B. 诱导“硬件预取” (The Sequential Access) 在微内核里，读取方式变成了：

```
__m256 b0 = _mm256_loadu_ps(B + k * ldb);
```

- **实现逻辑：**因为 `B + k * ldb` 在内存里是绝对线性连续的，代码上规避了 `B->data[k * cols + j]` 这种带大步长的原始索引。那种索引每跳一行就可能跨越一个内存页，导致 TLB 缓存失效。

关于内核大小，我们关注 float 单精度类型的内核大小，会发现 OPENBLAS 和 OPENBLIS（BLAS-like Library Instantiation Software）的内核大小不同。

调查发现：openblas 的 x86_64/ARM/RISCV 的内核大小都是 16 * 2/4/8，但是 OPENBLIS 的 x86_64 架构使用了 6 * 16/24 * 16 的大小，而 RISCV 居然是 7 * 64 的大小，ARM 架构是 2 * 10 这种十的倍数类型的大小。

那我们怎么选内核大小？本实验对比了不同内核尺寸（4x16, 6x16, 8x16）在不同矩阵规模下的表现。

V5 不同微内核尺寸性能对比 (单位: GFLOPS)

矩阵规模 (N)	4x16 内核	6x16 内核	8x16 内核
2048 x 2048	535.51	702.18	664.75
4096 x 4096	261.24	369.63	461.79
8192 x 8192	134.56	194.24	232.61
2400 x 2400	451.22	638.33	629.33

6*16 的缺点在于，一，无法整除 2048/4096/8192，剩下的 2 行（余数）不能进微内核（在 ALU 里），所以需要写逻辑判断，在 cpu 里做分支预测和跳转。二，寻址的时候，address = base + i *8192 。乘 4/8 直接逻辑左移就好，乘 6 需要调乘法器。

所以，这些优化都是权衡：AVX2 只有 16 个寄存器，8x16 需要 19 个。所以 CPU 必须把 3 个寄存器的值暂时寄存在内存栈（Stack）里。这本来会变慢。

不过不要纠结，我觉得主要了解一下不同内核大小的区别就好了，一切以具体的性能为主，OPENBLIS 选择 6*16 也是针对于具体的设备去做的选择。

3.3.5 V6: 二级缓存对齐与矩阵面板打包

1. V6 解决了什么核心问题？ 在 V5 之前，尽管我们使用了 SIMD 和寄存器阻塞，但 B 矩阵的访问依然存在巨大的隐患。

- **TLB 与非连续访存：**在计算 $C = A \times B$ 时，传统的行优先存储导致我们读取 B 的列时，地址在内存中是不连续的（跳跃步长为 N）。由于 B 的数据在内存中跨

度太大，CPU 无法有效地预取数据，导致计算单元（FMA）经常处于“空转”等待数据的状态。

- **多线程的私有化：**多线程环境下，我们要避免多个线程竞争同一块缓存行

2. 核心实现：pack_B_panel 的妙用 V6 引入了 Packing（打包）技术。把数据重排成连续的。

代码实现：

```
static inline void pack_B_panel(size_t k_len, size_t n_len, const
   float *src, size_t ld_src, float *dest)
{
    for (size_t k = 0; k < k_len; ++k)
    {
        memcpy(dest + k * n_len, src + k * ld_src, n_len * sizeof(
           float));
    }
}
```

- **实现原理：**该函数将 B 矩阵中原本分布在不同行、物理地址相隔甚远的元素，抽取出来并紧密地排列在 b_pack 缓冲区中。
- **结果：**在随后的微内核计算中，_mm256_loadu_ps(&b_pack[...]) 变成了完全的线性读取。这可以触发了 CPU 的硬件预取器（Prefetcher），让数据在计算需要之前就提前进入 L1 Cache。

3. 内存对齐与线程局部化 V6 改变了多线程环境下对内存的分配方式：

```
#pragma omp parallel
{
    float *b_pack = (float *)aligned_alloc(32, KC * NC * sizeof(float
       ));
    ...
    free(b_pack);
}
```

- **aligned_alloc(32, ...)：** AVX2 指令操作 256 位寄存器，要求地址最好是 32 字节对齐的。这消除了非对齐访存（Unaligned Access）带来的性能惩罚。
- **私有化缓冲区：**每个线程拥有独立的 b_pack。这意味着线程在打包和读取 B 时，完全不会发生总线冲突或伪共享，保证了多核心下的线性加速比。

4. 循环架构的重构： $jc \to kc \to ic$　V6 的三层分块（Tiling）顺序经过了精心设计，目的是最大化 L2 缓存命中率。

1. **外层 jc 循环：** 切分 N 维度，选定 B 的一个垂直面板（Panel）。
2. **中层 kc 循环：** 切分 K 维度，此时调用 `pack_B_panel`。
   - **关键点：** 一旦 B 的这个子块被打包进 `b_pack`（大小通常设为能放入 L2 缓存，如 512 KB），它就会在接下来的 ic 循环中被反复复用。
3. **内层 ic 循环：** 切分 M 维度，读取 A 的行块与已经在缓存里的 `b_pack` 进行计算。

物理意义：这种设计让 B 的数据最大程度放在 L2 缓存里，直到 A 的所有行都遍历一遍。

5. 6x16 寄存器阻塞微内核　在最内层，V6 使用了极致的寄存器压榨：

```
__m256 c0 = _mm256_setzero_ps(), c1 = _mm256_setzero_ps();
...
for (size_t k = 0; k < k_len; ++k) {
    __m256 b0 = _mm256_loadu_ps(&b_pack[k * n_len + j]);
    __m256 b1 = _mm256_loadu_ps(&b_pack[k * n_len + j + 8]);

    __m256 va0 = _mm256_set1_ps(A->data[(i + 0) * K + (kc + k)]);
    c0 = _mm256_fmadd_ps(va0, b0, c0);
    c1 = _mm256_fmadd_ps(va0, b1, c1);
    ...
}
```

- **寄存器阻塞：** 使用 12 个寄存器（$c0$ 到 cb）暂存结果，意味着在 k 循环中，我们完全不需要访问 C 矩阵。
- **计算密度：** 每次从 B 加载数据，都会被 A 的 6 个不同元素复用。访存比（Compute-to-Memory Ratio）显著提高。

3.4　架构级深度调优

从 v7 开始到 v10，从结果上，相比于 v6，基本都是负优化，只有 v10 可以跟 v6 一个水平，但还是比不过 openblas。

版本号	A 矩阵打包	B 矩阵打包	K 轴展开	软件预取	核心 SIMD 指令与寻址	核心优化目标 / 瓶颈解决
V6	无	有	无	无	_mm256_loadu_ps _mm256_set1_ps	B 矩阵连续化访存
V7	有	有	无	无	纯指针递增寻址	A/B 双侧连续化访存
V8	有	有	4 倍展开	有	_mm_prefetch	隐藏访存延迟 / 提升流水线并发
V9	无	有	无	无	_mm256_load_ps	适配 L2 缓存容量 / 减少打包开销
V10	有	有	无	无	_mm256_broadcast_ss	消除寄存器重排延迟 / 满载 FMA 吞吐

3.4.1 V7-V10: 连续性访存、预取与 SIMD 指令替换

1. V7：双重打包与对称连续性 优化逻辑：V6 仅打包了 B 矩阵（V6 分块参数设定为 MC=256，KC=256，其中 NC 从 512 扩展至 1024 以扩大 L2 缓存行覆盖面），V7 引入了对 A 矩阵面板的打包（pack_A_panel）。

A. A 矩阵面板重排代码：

```
static inline void pack_A_panel(int k_len, const float *src, int
   ld_src, float *dest) {
    for (int k = 0; k < k_len; ++k) {
        for (int i = 0; i < 6; ++i) {
            *dest++ = src[i * ld_src + k];
        }
    }
}
```

B. 微内核内部对称寻址代码：

```
// A 矩阵面板连续读取
const float *ap = &a_pack[k * 6];
c0 = _mm256_fmadd_ps(_mm256_set1_ps(ap[0]), b0, c0);
```

- **技术细节与优化原理：**在进入 6×16 微内核前，pack_A_panel 函数将 A 矩阵跨行、不连续的 6 行数据重新物理排列至对齐的连续缓冲区 a_pack 中。
- **效果：**这一重排使最内层计算循环对 A 的加载从带大跨步的物理寻址 A->data[(i+r)*K] 转换为 ap[r] 的线性递增寻址。此举彻底消除了 K 维主循环中的非连续地址跳转，显著提升了一级数据缓存（L1d）的缓存行（Cache Line）利用率。

2. V8：软件预取与 Lion Cove 吞吐压榨 优化逻辑：针对具有更宽指令发射宽度的处理器架构，引入 K 维度 4 倍展开，并配合显式软件预取指令。

C. 流水线深度展开与预取代码：

```
size_t k = 0;
for (; k + 3 < k_len; k += 4) {
    // 软件预取：提前拉取后续 B 块数据至 T0 缓存
    _mm_prefetch((const char*)&b_pack[(k + 8) * n_len + j],
       _MM_HINT_T0);

    for(int kk = 0; kk < 4; ++kk) {
        __m256 b0 = _mm256_loadu_ps(&b_pack[(k+kk) * n_len + j]);
        __m256 b1 = _mm256_loadu_ps(&b_pack[(k+kk) * n_len + j + 8]);
        const float *ap = &a_pack[(k+kk) * 6];

        c0 = _mm256_fmadd_ps(_mm256_set1_ps(ap[0]), b0, c0);
        c1 = _mm256_fmadd_ps(_mm256_set1_ps(ap[0]), b1, c1);
        // ... 展开累加至 cb
    }
}
```

- **技术细节与优化原理：** 通过将 K 维度的计算循环执行 4 倍手动硬编码展开，消除了大量的循环边界比较与条件跳转指令。同时，利用 `_mm_prefetch` 内置函数向处理器内核发送内存探针信号。
- **效果：** 4 倍展开极大地拓宽了编译器进行指令分派和重排的乱序调度空间（Instruction Scheduling）。而硬件探针提前 8 个迭代步长将后续处于主存中的 B 矩阵块连续拉入 L1 缓存中，使未来高频计算数据的加载延迟完全与当前正在执行的 FMA 乘加指令掩盖重叠。

3. V9：二级缓存参数特化与单打包决策 (matmul_v9_lunar_max) 优化逻辑：针对目标物理硬件特有的 2.5 MiB L2 大容量高速缓存，特化分块参数，测试非连续访存与内存拷贝（memcpy）开销之间的物理瓶颈对冲平衡点。

D. V9 缓存直接映射寻址代码（跳过 A 打包）：

```
// 不执行 A 的 memcpy，直接通过 set1 从原阵列中读取广播
__m256 va0 = _mm256_set1_ps(A->data[(i+0)*K+(kc+k)]);
c0 = _mm256_fmadd_ps(va0, b0, c0);
```

- **硬件容量特化对齐：** V9 版本将分块常数精确调整为：

```
#define MC_V9 512
#define KC_V9 512
#define NC_V9 1024
```

此时，打包后驻留在核心二级缓存中的 B 矩阵子块（b_pack）的物理内存占用量计算如下：

$$512\ (\text{KC_V9}) \times 1024\ (\text{NC_V9}) \times 4\ \text{Bytes} = 2,097,152\ \text{Bytes} = 2\ \text{MiB}$$

- **物理机制剖析：** 该分块大小恰好能够完美占满 Lion Cove P 核心私有的 2.5 MiB L2 缓存容量上限，且留有 512 KiB 的空间用于存放 A 矩阵数据、系统指令及栈空间。这一设计保证了 B 的数据子块在整个计算周期内始终处于“最热”的二级缓存状态，杜绝了由于分块过大导致的数据被挤出（Cache Eviction）。
- **单打包权衡决策：** 由于 B 矩阵已被极其紧密地锁定在 2 MiB 的 L2 缓存中，V9 尝试取消对 A 矩阵的运行时重排打包（即免去 pack_A 的内存拷贝开销），直接通过 _mm256_set1_ps 在原阵列中逐元素非连续读取并广播。实测表明，在特定矩阵规模下，消除 A 打包所节省的内存总线开销，成功对冲并超过了非连续访存的额外延迟。

4. V10：广播指令优化 (matmul_v10_final) 优化逻辑：在大规模计算场景下，V10 版本证实了双重对齐打包仍是防止 L1/L2 缓存频繁换入换出的核心护城河。因此，本版本回归了对称双打包结构，并利用替换了硬件广播指令。

```
// 回归 A 连续打包，并在微内核内调用原生广播指令
const float *ap = &a_pack[k * 6];
c0 = _mm256_fmadd_ps(_mm256_broadcast_ss(&ap[0]), b0, c0);
c1 = _mm256_fmadd_ps(_mm256_broadcast_ss(&ap[0]), b1, c1);
```

- **分块参数：** V10 版本的参数回归至更稳健的黄金分块比例：

```
#define MC_V10 256
#define KC_V10 256
#define NC_V10 512
```

在这种尺度下，线程私有缓冲区 A 和 B 的总大小计算如下：

$$A_{pack} : 256\ (\text{MC_V10}) \times 256\ (\text{KC_V10}) \times 4\ \text{Bytes} = 256\ \text{KiB}$$

$$B_{pack} : 256\ (\text{KC_V10}) \times 512\ (\text{NC_V10}) \times 4\ \text{Bytes} = 512\ \text{KiB}$$

二者总内存总计 768 KiB，确保在任意主流 x86_64 处理器架构下（即使在超线程 L2 缓存容量减半的环境下），计算数据集均能绝对稳定地锁定在核心本地。

- **SIMD 指令：** 微内核全面废弃了 _mm256_set1_ps（其在编译后会在寄存器之间产生多次数据搬移与数据洗牌），替换为 _mm256_broadcast_ss。在底层汇编中，该

指令对应原生的 vbroadcastss 指令，利用内存寻址后端的广播执行通道，直接在单周期内将 A 面板内的浮点标量广播加载至 YMM 寄存器的所有通道，彻底释放了算术逻辑单元（ALU）的数据重排压力，使硬件 FMA 发射管线达到极限饱和状态。

3.4.2 matmul_improved 架构选择与参数决策

最终确立的 matmul_improved 版本在计算核心（Micro-kernel）层面完全继承了 V10 的极致设计（12 个 YMM 寄存器阻碍与 _mm256_broadcast_ss 硬件广播），但在内存与打包策略上，针对云服务器环境进行了以下针对性优化：

1. **全局一次性打包**
   通过系统指令 free -h 测得云服务器物理主存为 **220 GiB**。在充足的内存空间支撑下，算法可以直接在并行区域外通过 aligned_alloc(64, ...) 预分配完整的全局打包缓冲区（A_packed 与 B_packed）。这种“一次性静态打包、高频线性复用”的策略，彻底消除了计算循环中由线程局部 malloc/free 带来的系统调用开销。

2. **针对小缓存的高效分块参数**
   为了防止计算数据集超出服务器核心较小的二级缓存(L2 Cache)物理上限,matmul_improved 配合全局打包，将核心几何分块参数特化校准为：

```
#define MC 120    // 对应微内核高度 6
#define NC 256    // 对应微内核宽度 16
#define KC 512    // 共享维度分块
```

   该参数（120/256/512）确保了当前活跃子块的工作集能够完全锁死在核心的私有缓存内，在最大化缓存命中率的同时，显著减轻了总线带宽的压力。

4 OPENBLAS 分析

4.1 运行时调用链

当我在我们的系统上调 openblas 时，发生了什么。

我们使用

```
cblas_sgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
                n, n, n, 1.0, A->data, n, B->data, n, 0.0, C->data, n
                    );
```

去调用 openblas 的 cblas_sgemm ，具体参数信息如下：

基础运算公式为 $C = \alpha AB + \beta C$ 。

参数名称	对应变量	含义与详细说明
Order	CblasRowMajor	矩阵存储布局：指定矩阵按“行优先”方式存储
TransA / TransB	CblasNoTrans	转置选项：指定在计算前不对矩阵 A 或 B 进行转置处理
M, N, K	size, size, size	矩阵维度：M 为 A 的行数，N 为 B 的列数，K 为 A 的列数
Alpha	1.0f	乘积系数 α：用于缩放矩阵乘积 AB 的标量
A / B	A.data / B.data	矩阵指针：指向存储矩阵 A 和 B 数据块的首地址
lda / ldb	size	矩阵主维步长：在行优先模式下，代表矩阵每一行实际占用的内存宽度
Beta	0.0f	累加系数 β：用于缩放结果矩阵 C 的初始值
C	C.data	结果矩阵指针：指向存储运算结果的矩阵 C 的首地址
ldc	size	结果矩阵主维步长：矩阵 C 的主维大小

我们在这里使用 deepwiki.com 辅助我们快速阅读分析 blas 库。openblas 对于 SGEMM 实际上分为五层实现：我们先查询我们的 model：

```
teddy@LAPTOP-L71LF0F4:\~\$ lscpu | grep -E "Family|Model"
Model name:                           Intel(R) Core(TM) Ultra 7 255H
Model:                                197
```

那么我们的 Arrow Lake-H 架构（Model 197 = 0xC5）会走以下路径：

```
case 12: //family 6 exmodel 12
    switch (model) {
        case 6: // Arrow Lake
            if(support_amx_bf16())
                return CORE_SAPPHIRERAPIDS;
            if(support_avx512_bf16())
                return CORE_COOPERLAKE;
            if(support_avx512())
                return CORE_SKYLAKEX;
            if(support_avx2())
                return CORE_HASWELL;
            if(support_avx())
                return CORE_SANDYBRIDGE;
            else
```

```
                return CORE_NEHALEM;
    }
```

cpuid_x86.c:2427-2442

所以最终的微架构是 CORE_HASWELL，属于 x86_64 架构下的 Intel Haswell 处理器系列。OpenBLAS 会自动检测 CPU 并选择对应的优化内核，并且支持动态架构编译，可在运行时选择最优实现。CPU 识别代码只在启动时执行一次，不是每次 SGEMM 都调用的。

OPENBLAS 运行时调用链共 5 层：

表 1: GotoBLAS 架构分层设计与职责汇总

层次	**名称**	**文件**	**核心职责**
第 1 层	接口层	`interface/gemm.c`	参数校验、路径选择、缓冲区分配、线程数决策
第 2 层	多线程调度层	`driver/level3/level3_thread.c`	2D 分区，将矩阵切分给各线程
第 3 层	分块驱动层	`driver/level3/level3.c`	GotoBLAS 三层分块 (N/K/M 方向循环)
第 4 层	Pack 层	`kernel/x86_64/sgemm_ncopy_*.c` 等	矩阵重排 (ICOPY/OCOPY)，消除 stride 访问
第 5 层	汇编内核层	`kernel/x86_64/sgemm_kernel_16x4_haswell.S`	AVX2 FMA 计算，16x4 寄存器分块

那我们以 Intel Core Ultra 7 255H 设备，16 核全开启，8192 大小的矩阵为例：

第一层：入口与路径选择文件：interface/gemm.c

```
sgemm_() / cblas_sgemm()
|-- 参数校验
|-- 退化检查
|-- [路径1] 小矩阵检查: GEMM_SMALL_MATRIX_PERMIT()
|-- [路径2] SGEMM_DIRECT 直接路径
|-- 分配工作缓冲区: blas_memory_alloc(0)
|-- 多线程决策: get_gemm_optimal_nthreads(MNK)
`-- [路径3] 标准 GotoBLAS 路径 <- 8192x8192 走此路径
```

第二层：CPU 识别与内核选择文件：cpuid_x86.c

```
CPUID 检测
|-- Extended Model = 12 (0xC)
    `-- Model = 6 (Arrow Lake) -> CORE_HASWELL
```

第三层：多线程分区文件：driver/level3/level3_thread.c

```
sgemm_thread_nn()
```

```
|-- 2D 分区策略
|-- 每个线程负责: 约 2048 行 x 2048 列 的子矩阵
`-- 各线程并行调用 sgemm_nn() (单线程驱动)
```

第四层：GotoBLAS 三层分块文件：driver/level3/level3.c，函数 sgemm_nn()

```
sgemm_nn() (单线程, 处理 8192x8192 子矩阵)
|-- l2size = GEMM_P x GEMM_Q = 768 x 384 = 294,912 个 float
|-- [外层] N 方向循环
|    `-- [中层] K 方向循环
|         |-- ICOPY_OPERATION() (pack A)
|         `-- [内层] N 子方向循环
|              |-- OCOPY_OPERATION() (pack B)
|              `-- KERNEL_OPERATION() (计算内核)
`-- [M 方向循环] 剩余 M 分块
```

第五层：汇编计算内核文件：kernel/x86_64/sgemm_kernel_16x4_haswell.S

```
sgemm_kernel_16x4_haswell (计算 C[768x12] += A[768x384] x B[384x12])
|-- 寄存器分配
|-- 内层循环 (K=384 次迭代)
`-- 写回阶段 (SAVE16x4)
```

4.2 数据流模型与多级缓存映射机制

数据流汇总

```
主内存 (3x256 MB = 768 MB)
  A[8192x8192]  B[8192x8192]  C[8192x8192]
  L3 cache (约24 MB, Arrow Lake-H)
  L2 cache (2.5 MB, Lion Cove P核)
  L1 cache (48 KB, Arrow Lake-H)
  AVX2 寄存器 (ymm0~ymm15, 各256位)
```

关键数字（8192x8192，Haswell 内核）

维度	分块大小	迭代次数
N 方向	GEMM_R = 4096	2 次
K 方向	GEMM_Q = 384	22 次
M 方向	GEMM_P = 768	11 次
N 子方向	3 x UNROLL_N = 12	342 次
总计算量	—	1.1 TFLOPS

4.3 本阶段实现与标杆库的工程差异对比

这是在 openblas 里计算的全流程。那么我们能从 openblas 中学到什么呢？通过比较我的代码和 openblas，我总结了四个不同：

1. 编译器指令重排 vs. 纯手工汇编 - 我们使用了 _mm256_load_ps 和 _mm256_fmadd_ps。这看起来很底层，但实际上你还是将指令调度和寄存器分配交给了 GCC 编译器。GCC 很难完美预测 AVX2 乘加指令的延迟槽往往会导致数据还没加载完，FMA 指令就在流水线里干等 - 而 OpenBLAS (sgemm_kernel_16x4_haswell.S)：完全是纯手工编写的汇编代码。他们不用编译器，手写汇编，对于时钟周期和指令排布，例如 vmovups (加载) 和 vfmadd231ps (计算) 的交替，肯定是比 GCC 更适合计算，这我学不来

2. 内核的大小（6×16 vs. 16×4）我选择了 6×16 的微内核，而 OpenBLAS 选择了 16×4。理论上我有效利用的寄存器的数量更多，但是整体性能不佳，但性能不佳也不一定是这个原因，毕竟影响因素太多了。(越到后面做的很多都是负优化)

3. 多线程 (1D OpenMP vs. 2D Thread Grid) - 我使用 #pragma omp for collapse(2)，这是一种基于循环迭代次数的自动划分。它无法保证物理核心之间完美共享 L3 缓存。- OpenBLAS (sgemm_thread_nn)：采用了精心设计的 2D 网格分区（例如 16 线程划分为 4×4）。这种划分强制让同一列的 4 个线程共享 L3 缓存中的同一个 B 矩阵子块，极大地降低了主存的带宽压力。

4. JIT 与打包 - 我使用 omp parallel 提前把整个 A 和 B 矩阵提前打包了，居然忘记了我们 project2 做了那么多详细的 JIT 的分析！- OpenBLAS (ICOPY / OCOPY)：采用了 JIT Packing。它只在内层循环即将需要某个块的前一刻，才去 pack 那个刚好塞满 L2 缓存的 A 子块和塞满 L1 缓存的 B 子块。计算完立刻丢弃，复用打包缓冲区。

5 JIT 版本

5.1 操作系统与物理内存调优

那么我们学着 openblas，继续做优化我们现在在 56 核，两个 NUMA 节点的 Intel(R) Xeon(R) CPU E5-2680 v4 @ 2.40GHz 上。我们采用这个架构：

```
[Main Thread]
 `-- OMP 并行区域启动 (56 线程)
      |-- 每个线程仅分配几 MB 的本地 A_buffer 和 B_buffer (零内存峰
          值, 防 TLB Miss)
      `-- 循环 jc (OMP 分发 N 列块)
           `-- 循环 kc (K 维度分块)
                |-- JIT 线程内打包 B 矩阵块 (L3 缓存)
                `-- 循环 ic (M 维度分块)
                     |-- JIT 线程内打包 A 矩阵块 (L2 缓存)
```

```
                    `-- 调用 6x16 内核
```

从 openblas 中学到的具体优化有：1. 不要一次打包所有 AB 矩阵

```
float *local_A = (float*)_mm_malloc(GOLD_MC * GOLD_KC * sizeof(float)
    , 64);
float *local_B = (float*)_mm_malloc(GOLD_KC * GOLD_NC * sizeof(float)
    , 64);
```

为每个线程分配了 120KB 的 A 缓存和 512KB 的 B 缓存。算到哪一块，就在内层循环 pack_A_JIT 现打包哪一块。降低使用的内存峰值，避免 TLB。

5.2 多核并发调度与同步消除

2. 从独立访存到单一线程打包，其他线程共享 - 我们之前使用 #pragma omp for collapse(2) 。但是在超大矩阵场景下，所有线程独立执行数据读取和打包操作，导致内存总线请求量呈线性倍增，极易达到内存带宽上限。- 优化：通过 #pragma omp single 指派单一线程将公共数据块（矩阵 B 的分块）打包至共享内存池，所有线程复用该共享数据。

```
#pragma omp single
{
    pack_B_JIT(actual_kc, actual_nc, &B->data[kc*N + jc], N, shared_B
        );
}
```

6 最终性能测试与对标验证

6.1 全规模矩阵执行耗时与浮点运算性能表现

成果展示：使用 gcc -Ofast -march=native -fopenmp main.c matrix.c matmul_legacy.c gemm_jit.c -o gemm_bench -L$CONDA_PREFIX/lib -I$CONDA_PREFIX/include -lopenblas -Wl,-rpath=$CONDA_PREFIX/lib 编译使用 alias run_bench='numactl –interleave=all ./gemm_bench' 分配双节点运行使用 run_bench 启动

6.2 对标基准库加速比分析

OpenBLAS 对比加速比 (Speedup)

表 2: 矩阵乘法性能测试结果汇总 (Time & GFLOPS)

矩阵规模 ($N \times N$)	指标	Plain (Benchmark)	Improved (SIMD/OMP)	JIT (Generated)	OpenBLAS
16×16	Time (s)	0.000011	0.000055	0.000012	0.000079
	GFLOPS	0.75	0.15	0.69	0.10
128×128	Time (s)	0.002228	0.000284	0.000289	0.000239
	GFLOPS	1.88	14.77	14.50	17.54
1024×1024	Time (s)	2.293088	0.004810	0.004444	0.004692
	GFLOPS	0.94	446.45	483.22	457.68
8192×8192	Time (s)	Skipped	0.886784	0.810066	0.908520
	GFLOPS	-	1239.89	1357.31	1210.22
16384×16384	Time (s)	Skipped	6.964683	6.833305	6.681441
	GFLOPS	-	1262.96	1287.24	1316.50
32768×32768	Time (s)	Skipped	52.412215	54.129841	41.859317
	GFLOPS	-	1342.60	1300.00	1681.08
65536×65536	Time (s)	Skipped	408.587450	424.037382	339.170381
	GFLOPS	-	1377.80	1327.60	1659.79

矩阵规模	OpenBLAS vs Improved	OpenBLAS vs JIT
16 x 16	0.69 x	0.15 x
128 x 128	1.19 x	1.21 x
1024 x 1024	1.03 x	0.95 x (JIT 胜出)
8192 x 8192	0.98 x	0.89 x (JIT 胜出)
16384 x 16384	1.04 x	1.02 x
32768 x 32768	1.25 x	1.29 x
65536 x 65536	1.20 x	1.25 x

7 低时间复杂度算法

7.1 Strassen 算法 ($O(N^{2.807})$)

Strassen 算法是历史上第一个在理论上超越 $O(N^3)$ 的矩阵乘法算法。其核心突破在于通过增加矩阵加法的次数，换取了计算开销最昂贵的乘法次数的减少。

7.1.1 分治与子块定义

标准的分治法将 $N \times N$ 的矩阵 A 和 B 各划分为 4 个 $(N/2) \times (N/2)$ 的子矩阵：

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$$

7.1.2 构造 7 个中间乘积项 ($P_1 \sim P_7$)

Strassen 巧妙地构造了 7 个中间矩阵，每个矩阵仅需 1 次递归乘法：

- $P_1 = A_{11} \times (B_{12} - B_{22})$
- $P_2 = (A_{11} + A_{12}) \times B_{22}$
- $P_3 = (A_{21} + A_{22}) \times B_{11}$
- $P_4 = A_{22} \times (B_{21} - B_{11})$
- $P_5 = (A_{11} + A_{22}) \times (B_{11} + B_{22})$
- $P_6 = (A_{12} - A_{22}) \times (B_{21} + B_{22})$
- $P_7 = (A_{11} - A_{21}) \times (B_{11} + B_{12})$

7.1.3 结果子块的组合还原

利用这 7 个 P 项，通过纯加减法操作即可还原出结果矩阵 C：

- $C_{11} = P_5 + P_4 - P_2 + P_6$
- $C_{12} = P_1 + P_2$
- $C_{21} = P_3 + P_4$
- $C_{22} = P_5 + P_1 - P_3 - P_7$

7.1.4 复杂度与工程权衡

其递归式为 $T(N) = 7T(N/2) + O(N^2)$，由主定理可得 $T(N) = O(N^{\log_2 7}) \approx O(N^{2.807})$。**注意**：虽然乘法减少了，但算法引入了 18 次加减法，且破坏了内存访问的连续性。

7.2 Winograd 变体

Winograd 在 Strassen 的基础上进一步优化了加法的执行效率，是目前高性能库中处理子立方算法时的首选变体。

- **原理**：Winograd 通过重新排列代数式，将矩阵加/减法的次数从 18 次减少到了 15 次。
- **优势**：尽管理论复杂度阶数相同，但减少 3 次加法意味着在大规模运算（如 8K 或 64K）中能显著降低访存压力。

7.3 理论极限：Laser Method 与 Coppersmith-Winograd

在学术研究领域，学者们利用“激光法”（Laser Method）不断刷新理论复杂度的上限。

- **Coppersmith-Winograd 算法**：通过对张量（Tensor）的复杂变换，将复杂度降至约 $O(N^{2.375})$。
- **现状**：目前的理论极限约为 $O(N^{2.371})$。
- **评价**：此类算法由于常数项系数过于庞大，被称为“银河算法”，目前尚无法在现实硬件（如 X86 或 ARM 平台）中超越 OpenBLAS 等成熟库的性能。

7.4 不同算法性能实测与复杂度验证

7.4.1 编译环境说明

采用了以下编译选项：

Listing 1: 编译命令

```
gcc -O3 -march=native -fopenmp main.c matrix.c -o matmul_bench -
   lopenblas
```

7.4.2 实验数据汇总

下表记录了朴素算法（Plain）、Strassen 算法与 Winograd 变体在不同规模下的执行耗时：

表 3: 不同算法执行时间对比（单位：秒）

矩阵规模 ($N \times N$)	**Plain** ($O(N^3)$)	**Strassen** ($O(N^{2.807})$)	**Winograd** ($O(N^{2.807})$)
16×16	0.00000 s	0.00000 s	0.00000 s
128×128	0.00013 s	0.00011 s	0.00010 s
1024×1024	0.07346 s	0.07799 s	0.07493 s
8192×8192	106.38351 s	19.97396 s	21.32087 s

8 GPU 加速优化与性能分析

8.1 实验环境说明

本实验在高性能计算服务器上完成，硬件配置及软件环境如表 4 所示。为了确保测试结果的准确性并遵守实验室算力使用规范，实验通过 `CUDA_VISIBLE_DEVICES=0` 环境

变量显式指定使用服务器中的 0 号 NVIDIA TITAN V 显卡。该显卡基于 Volta 架构，拥有 5120 个 CUDA 核心及 12GB HBM2 显存，单精度浮点（FP32）理论峰值性能约为 14.9 TFLOPS。

表 4: 实验硬件与软件环境配置

组件	配置规格
中央处理器 (CPU)	2 × Intel Xeon E5-2680 v4 (共 28 物理核 / 56 逻辑线程)
图形处理器 (GPU)	**NVIDIA TITAN V (Device 0)**
显存 (VRAM)	12GB HBM2
操作系统	Ubuntu 22.04 LTS
编译器	NVCC (CUDA 13.0) / GCC 11.4
数学库	cuBLAS v12.x / OpenBLAS v0.3.x

8.2 CUDA 核心优化实现

GPU 版本的优化思路继承了 CPU 阶段的“分块（Tiling）”与“打包（Packing）”思想，并将其映射到 GPU 的存储层次结构中。

8.2.1 共享内存分块 (Shared Memory Tiling)

为了规避频繁访问高延迟的全局显存（Global Memory），算法利用片上高速共享内存（Shared Memory）充当二级缓存。通过 `__shared__` 关键字定义的 `sA` 和 `sB`，实现了数据的协同加载（Coalesced Loading），将访存压力降低了 $1/BM$。

8.2.2 寄存器分块 (Register Tiling)

每个线程维护一个 8×8 的寄存器阵列 `rC`，用于存储中间累加结果。这种设计最大化了浮点运算单元（FMA）的吞吐量，减少了对共享内存的读写频率，其核心微内核逻辑如下：

Listing 2: CUDA 极致优化内核代码片段

```
// 每个线程负责 8x8 的计算战区
float rC[TM][TN] = {0.0f};
float rA[TM], rB[TN];

// K 维度外层循环
for (int k = 0; k < K; k += BK) {
    // 协同搬运数据至共享内存 sA, sB
    __syncthreads(); // 隐式屏障：确保加载完成
```

```
    // 寄存器级微内核计算
    for (int dotIdx = 0; dotIdx < BK; ++dotIdx) {
        for (int i = 0; i < TM; ++i) rA[i] = sA[(tidy * TM + i) * BK
           + dotIdx];
        for (int i = 0; i < TN; ++i) rB[i] = sB[dotIdx * BN + (tidx *
            TN + i)];

        for (int i = 0; i < TM; ++i) {
            for (int j = 0; j < TN; ++j) {
                rC[i][j] += rA[i] * rB[j]; // 寄存器级 FMA
            }
        }
    }
    __syncthreads(); // 确保当前块计算完成再进入下一轮加载
}
```

8.3 编译与运行指令

由于本实验环境基于 Conda 进行无 sudo 权限的库管理，编译时需手动指定路径。通过 -Xcompiler 参数将 CPU 端的优化指令（如 -march=native）传递给底层 GCC 编译器。

```
# 编译命令
nvcc -O3 -Xcompiler "-fopenmp -march=native" \
     main.c matrix.c matmul.c matmul_jit.c matmul_cuda.cu \
     -o gemm_cuda_bench \
     -L$CONDA_PREFIX/lib -I$CONDA_PREFIX/include \
     -lopenblas -lcudart -lcublas

# 运行命令：指定 0 号显卡并执行 NUMA 优化
CUDA_VISIBLE_DEVICES=0 numactl --interleave=all ./gemm_cuda_bench
```

8.4 测试结果与分析

表 5 汇总了各算法在不同矩阵规模下的吞吐性能（单位：GFLOPS）。测试结果显示，由于 PCIe 总线传输开销，GPU 在小规模矩阵（$N \leq 128$）下性能极低；但在大规模矩阵（$N \geq 8192$）下，CUDA 展现了压倒性的算力优势。

表 5: 各算法性能 (GFLOPS) 全维度对比测试结果

规模 (N)	**CPU JIT**	**OpenBLAS**	**Custom CUDA**	**cuBLAS**
16	1.68	0.12	0.00	0.00
128	23.21	1.53	1.50	0.92
1024	193.17	225.37	209.34	506.09
8192	1320.02	1558.61	9081.90	8767.58
16384	1222.26	1448.22	18917.69	19146.76
32768	1080.79	1562.32	**54459.70**	**56550.47**

9 ARM 架构下的高性能 GEMM 移植与极限优化

9.1 实验环境与硬件规格

为了全面评估矩阵乘法（GEMM）在非 x86 架构上的可移植性与性能表现，本实验在阿里云上部署了企业级高性能 ARMv9-A 服务器实例。该平台采用专为高并发、高算力云工作负载设计的 **ARM Neoverse-N2** 核心。该实例在硬件层面上表现出极强的高并发并行计算性能，配备 32 个独占的物理核心，且支持先进的矢量扩展指令集。具体的实验硬件及编译环境规格如表 6 所示。

表 6: **ARM Neoverse-N2 实验平台硬件规格与系统环境**

硬件与系统参数	**详细配置规格**
CPU 架构	aarch64 (64-bit ARMv9-A)
处理器微架构	ARM Neoverse-N2
CPU 核心数	32 物理核心 (单插槽，1 线程/核心，无超线程干扰)
主频 (Clock Speed)	3.0 GHz (恒定主频)
L1d / L1i 缓存	每核心独占 64 KiB (总计 2 MiB 运行指令 / 2 MiB 运行数据)
L2 缓存 (Private L2)	每核心独占 1 MiB (总计 32 MiB)
L3 缓存 (Shared L3)	64 MiB 共享系统级缓存 (System-Level Cache)
矢量扩展指令集	fp, asimd (128-bit NEON), sve, sve2, i8mm, bf16
物理系统内存	64 GiB DDR5 (系统实际可用约 60 GiB)
操作系统	Ubuntu 24.04 LTS (Linux 内核版本: 6.8.0-aarch64)
编译器工具链	GCC version 13.3.0 (支持 OpenMP 4.5 并发调度)
外部数学库	OpenBLAS version 0.3.26 (针对 aarch64 极致调优版本)

9.2 从 x86 AVX2 到 ARM NEON 的 SIMD 矢量化重构

在将原有基于 x86 AVX2（256 位寄存器位宽）的手写高性能 GEMM 内核迁移至 ARM 架构时，最核心的挑战在于 SIMD 矢量化指令集的重构。

x86 的 AVX2 拥有 16 个 256 位宽的 YMM 寄存器，单条矢量指令可以并行处理 8 个单精度浮点数；而 ARM NEON 矢量寄存器位宽为 128 位，单条指令可处理 4 个单精度浮点数。虽然单矢量单次计算的数据位宽减半，但 ARM 架构提供了一个至关重要的硬件红利：**拥有 32 个物理矢量寄存器 (v0–v31)**，是 x86 架构的两倍。这一特性为我们在微内核层面展开更大维度的计算、减少临时内存读写（Register Spills）提供了坚实的硬件保证。

9.2.1 6 × 16 内核的分配

我们沿用 **6 × 16** 乘积微内核：

- **累加器寄存器 (C 矩阵局部块):** 在微内核中，16 列的数据需要 4 个 128 位 NEON 寄存器承载（16×32 bits $= 512$ bits $= 4$ 向量寄存器）。为了支持 6 行计算，共需要 $6 \times 4 = \mathbf{24}$ 个矢量寄存器（代码中命名为 `c00` 到 `c53`）作为累加器，死死锁定中间值，避免反复写入内存。
- **B 矩阵数据加载器:** 为了能够在每个迭代中流畅地进行 FMA 运算，我们需要 **4** 个寄存器（`b0` 到 `b3`）一次性载入 16 个连续的 B 矩阵列元素。
- **A 矩阵数据广播器:** 每次迭代中，我们只需加载 A 矩阵的一个标量元素并将其广播至整个向量通道中，这需要 **1** 个寄存器（`a`）。

通过上述配置，热点内层循环累计占用了 $24 + 4 + 1 = \mathbf{29}$ 个 NEON 寄存器。这在完美利用 ARM 寄存器池的同时，还留有 3 个寄存器进行地址指针移动等辅助指令，最大程度规避了寄存器溢出导致的栈帧拷贝，极大地提升了指令级流水线的吞吐量。

x86 AVX2 与 ARM NEON 指令级的直接映射关系如表 7 所示。

表 7: **x86 AVX2 与 ARM NEON SIMD 关键指令映射表**

SIMD 功能定义	x86 AVX2 Intrinsic	ARM NEON Intrinsic
矢量数据类型	`__m256`	`float32x4_t`
累加器初始化 (0.0)	`_mm256_setzero_ps()`	`vdupq_n_f32(0.0f)`
连续内存加载 (对齐)	`_mm256_load_ps(ptr)`	`vld1q_f32(ptr)`
标量数据广播	`_mm256_broadcast_ss(ptr)`	`vld1q_dup_f32(ptr)`
融合乘加 (FMA)	`_mm256_fmadd_ps(a, b, c)`	`vfmaq_f32(c, a, b)`
矢量相加	`_mm256_add_ps(a, b)`	`vaddq_f32(a, b)`
连续内存存储 (非对齐)	`_mm256_storeu_ps(ptr, v)`	`vst1q_f32(ptr, v)`

9.3 实验结果与性能评估

表 8: **ARM Neoverse-N2 平台下矩阵乘法性能评测结果 (时间: 秒 / 算力: GFLOPS)**

矩阵维度 ($N \times N$)	Plain C		Improved (NEON)		OpenBLAS	
	Time (s)	GFLOPS	Time (s)	GFLOPS	Time (s)	GFLOPS
16×16	0.000002	3.79	0.008370	0.00	0.000039	0.21
128×128	0.001309	3.20	0.005298	0.79	0.001169	3.59
1024×1024	1.641778	1.31	0.003210	669.09	0.002805	765.55
8192×8192	Skipped	Skipped	0.973784	1129.11	0.873784	1258.33
16384×16384	Skipped	Skipped	7.695756	1142.98	7.100615	1238.78
32768×32768	Skipped	Skipped	63.827944	1102.48	61.590441	1142.53

10 未来工作与不足

1. **不同编译器版本**
   本项目使用的是 GCC 13。五一的时候 GCC16 都出来了。其内部的循环展开、汇编指令重排以及寄存器分配都略有区别。https://gcc.gnu.org/gcc-16/changes.htmlopenmp

2. **RISC-V 架构移植与向量化（RVV）性能评估**
   我的 RISCV 开发板 (香蕉派 F3，8 核) 五一结束了才到，写这个 project 的时候串口还在调试，没能实现。

   - **RISC-V Vector (RVV 1.0) 实现：**针对支持向量扩展（Vector Extension）的 RISC-V 硬件平台，使用 RVV 1.0 汇编或内置函数（Intrinsics）重构微内核。
   - **向量长度无关性（VLA）调优：**利用 RISC-V 独特的向量长度无关（Vector Length Agnostic）编程特性，探索免除硬编码分块尺寸的通用高性能微内核拓扑，并与 x86 平台上硬编码物理寄存器阻塞（YMM 限制）的性能表现进行深入的定量对比分析。

3. **分块参数与微内核尺寸**
   目前选定的分块尺寸（如 MC, NC, KC）与 6×16 微内核结构是其实不一定是最好的，也没有做非常详细的调试。

4. **不同计算库的比较**
   Intel 有 MKL 库：https://www.intel.cn/content/www/cn/zh/developer/tools/oneapi/onemkl-download.html?operatingsystem=windowswindows-install=offline 这个也是边打包

边运算的 (JIT)，而且对于小矩阵也有比较好的优化，不过时间原因没能对比分析了。

5. **用 Vtune/perf 做采样**
服务器没有 sudo 权限，插探针稍微有点麻烦，然后本地的版本又多，所以懒了。我上一个 project 用探针分析的比较多，这个 project 如果能结合具体的截图，比如 TLBmiss 多少，L1/2/3Miss 多少，会更有说服力一些。这个 project 就只用了时间和 GFlops。

11 总结与心得

我认为这个 project 的矩阵乘法，其实很多地方都可以优化。这延续了上一个 project 的性能分析：我们的代码到最终的输出，究竟要经过几步？预处理，编译，汇编，链接，运行时的动态链接，多线程，I/O。每一步能优化的都太多了。而且，优化不是 1+1=2 的，很可能 1+1<2，可能每个优化看上去都是好的，但是加起来会让性能变慢。所以，从 v6 之后，我都不太敢轻易下结论，我们的性能瓶颈究竟在哪，因为没有做消融实验，真不好说。通过 project3 收获：(有硬件换硬件，有多核换多核，有 GPU 就换 GPU，金钱才是性能瓶颈。) 我们能通过具体的步骤，一步一步做优化，让我对硬件有了更多的理解。此外，感受到了 x86 和 arm 的不同，还有 GPU 的强大。我们一开始从绑定正确的 openblas 库，然后一个一个版本做优化，学习 openblas 库，模仿并使用 JIT，然后在 GPU 和 ARM 架构的 CPU 测试。基本上没有几个能跑赢 Openblas 的，不过通过这样的 project，确实收获良多。

完整代码 (包括 ARM，CUDA 版本) 详见：https://github.com/Teddy-Yangjiale/Sustech-CS219-Advanced-Programming